

COMP 424 - Artificial Intelligence

Local search for optimization

Lecture 4: September 14th, 2026

Su Lin Blodgett (sulin.blodgett@mcgill.ca)

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Announcements

- Midterm has been scheduled
 - October 22nd, 7:00pm – 8:30pm in McMed
- Assignment preview sessions

Recap: Informed Search

- Using intuition about a state's distance to the goal: a **heuristic h**
- Two new (and one old) algorithms
 - **Uniform cost search:** greedy with respect to $g(n)$, cost of path from the start to n
 - **Best-first search:** greedy with respect to $h(n)$, estimate of path cost from n to the goal
 - **A* search:** greedy with respect to $f(n) = g(n) + h(n)$
 - Considers both backward and forward cost

Properties of A*

- A* is optimal if the heuristic is **admissible**
- **Admissibility:** The heuristic is always 'optimistic'
 $h(s) \leq h^*(s) \forall s$, where $h^*(s)$ is the true cost to the nearest goal
 - i.e., it never overestimates
- We also want an h that offers as tight a bound as possible

Properties of A*

- Performance depends strongly on the heuristic chosen

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(n)$: number of misplaced tiles = 8 for start state
 - admissible: any tile out of place must be moved at least once
- $h_2(n)$: Manhattan distance = 18 for start state
 - also admissible

Properties of A*

- Performance depends strongly on the heuristic chosen

	Search Cost (nodes generated)		
d	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6
4	112	13	12
6	680	20	18
8	6384	39	25
10	47127	93	39
12	3644035	227	73
14	—	539	113
16	—	1301	211
18	—	3056	363
20	—	7276	676
22	—	18094	1219
24	—	39135	1641

Test over 1200 random problems, solution lengths 2 to 24

Properties of A*

- Performance depends strongly on the heuristic chosen
- Practical drawback: space complexity (why?)
- But still widely used today (e.g., video game pathfinding)

Today: Local search for optimization

Uninformed search

- Assumes **no knowledge** about the problem.
- BFS, DFS, Iterative deepening

Informed search

- Use **knowledge about the problem**, in the form of a heuristic.
- Best-first search, heuristic search, A* (and extensions)

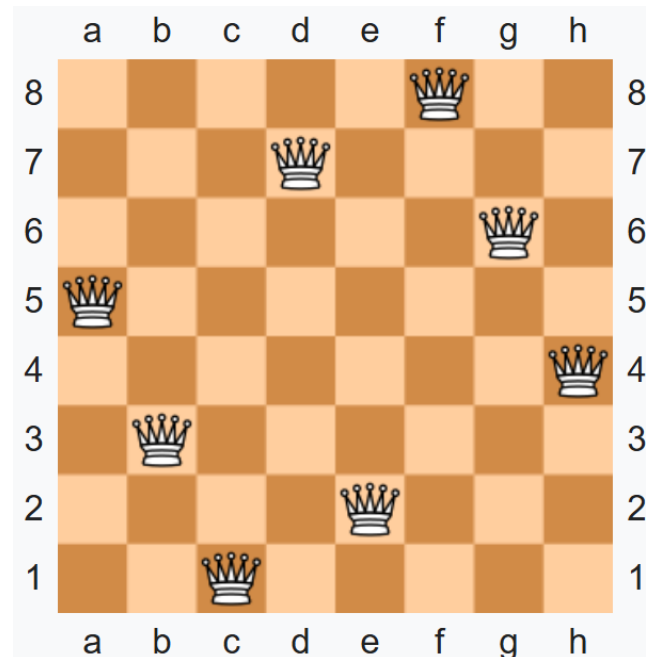
Search for optimization problems:

- Search over large-dimensional spaces
- Iterative improvement algorithms:
 1. Hill climbing
 2. Simulated annealing
 3. Parallelism and beam search
 4. Genetic algorithms

Today!

Search and optimization

1. **Search so far:** solution is a *path*, and each state is potentially on the optimal path
2. **Search today:** we are less interested in the path to the goal, and each state is a potential solution (an assignment of values to variables)



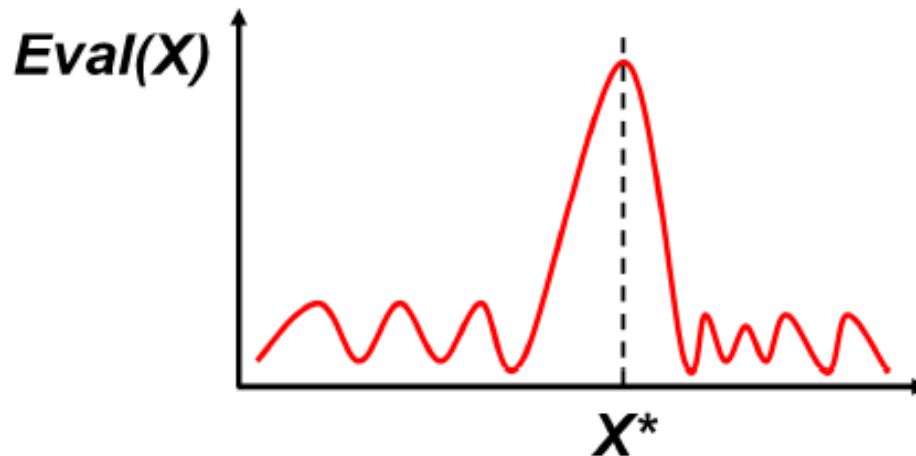
Eight queens problem

Search and optimization

1. **Search so far:** solution is a *path*, and each state is potentially on the optimal path
2. **Search today:** we are less interested in the path to the goal, and each state is a potential solution (an assignment of values to variables)
 - Exam scheduling – final exam schedule that minimizes conflicts

Search and optimization

- So far:
 - Finding the solution with the minimal cost, where the cost is the sum of edge weights (e.g., A* search)
- Today:
 - Case where solution cost is some arbitrary function ($Eval(X)$)
 - Want to find best solution (X^*) – **optimization problem**
 - best state according to $Eval(X)$



Optimization problems are everywhere

Scheduling

- **Given:** a set of tasks to be completed, with durations and mutual constraints (e.g. task ordering, joint resources)
- **Goal:** generate shortest schedule (assignment of start times to tasks)

User customization

- **Given:** customers described by characteristics (age, gender, location, etc.) and previous purchases
- **Goal:** find a function from characteristics to products that maximizes expected gain

Generating text from a language model

- **Given:** a trained LLM, a linguistic context, a vocabulary
- **Goal:** find the most appropriate sentence to generate in that context

Local search strategy

Contrast with search from last lecture!

- 1. Constructive methods:** Start from scratch and build up a solution.
 - Searching *systematically* – keeping one or more paths in memory, recording alternatives that have been explored
 - This is the type of method we have seen so far
- 2. Iterative improvement/repair methods:** Start with a solution (which may be broken / suboptimal) and improve it!

General intuition for today

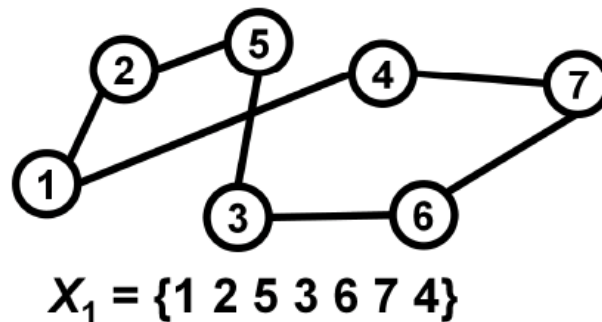
- Start from an initial solution, possibly very bad or incorrect
 - e.g., Just schedule all the exams arbitrarily at some time within the exam period
- Keep changing the solution to gradually improve it
 - e.g., Keep swapping exam time slots around to minimize the number of conflicts

Characteristics

- An optimization problem is described by:
 - A set of **states** (= **configurations**)
 - An **evaluation function**
- For interesting optimization problems:
 - The state space is *too big* to enumerate all states
 - Or, the evaluation function is *too expensive* to compute for all states
- In iterative improvement, *a state is a candidate solution!*
 - It might be a partial or incorrect solution; if so, should be reflected in evaluation function.

Travelling Salesman Problem (TSP)

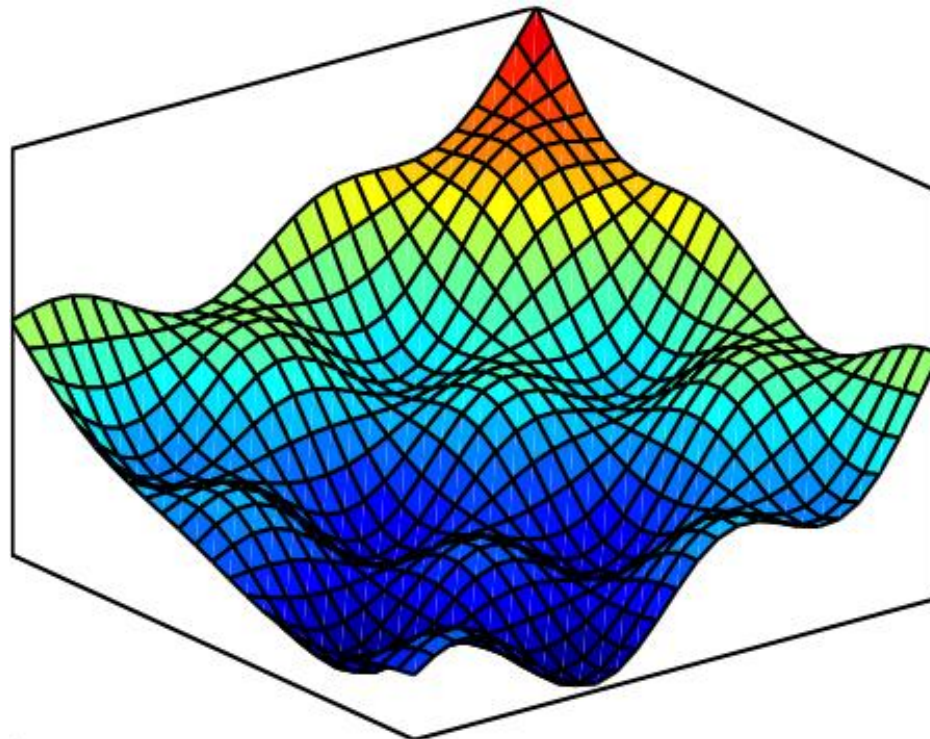
- **Given:** a set of vertices and distance between pairs.
- **Goal:** construct the shortest path that visits every vertex exactly once (a **tour**)
 - e.g., consider these seven cities on a map



- A state in the iterative improvement paradigm is a candidate tour; the evaluation function would be length of the tour.
 - X_1 (above) is a tour
 - Provably very hard (NP-complete) to find the **best** solution

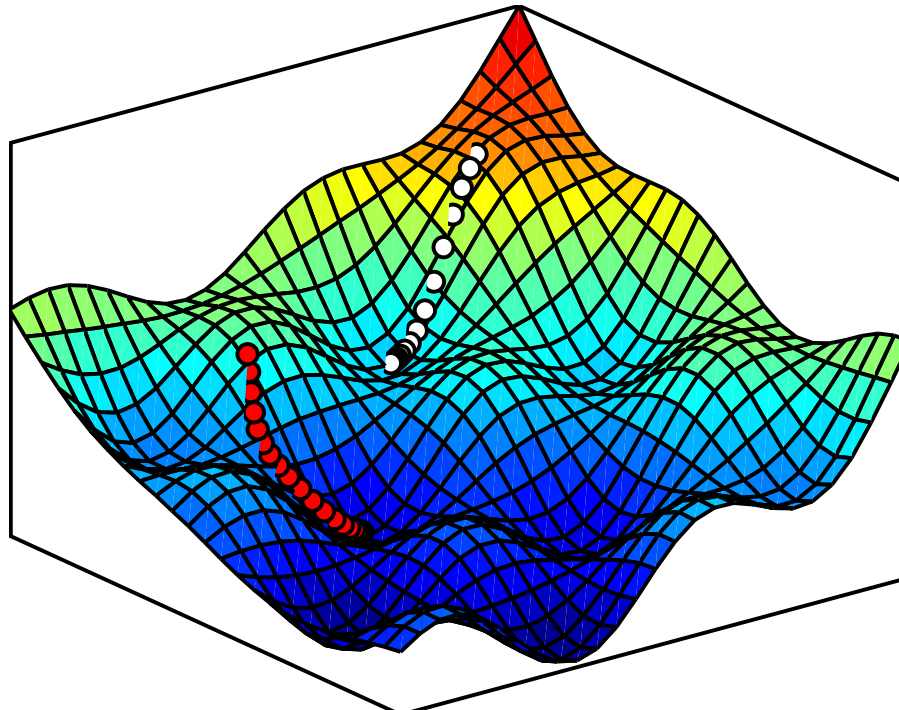
Visualizing iterative improvement

- **An example:**
 - Consider all possible solutions laid out on a landscape.
 - Suppose we want to find the lowest point.
 - Unlike TSP, the space of solutions here is continuous



A generic local search algorithm

- Start from an initial state x_0 .
- Repeat until satisfied:
 - Generate the set of neighbours of x_i and evaluate them.
 - Select one of the neighbours, x_{i+1} .
 - The selected neighbor becomes the new state.



A generic local search algorithm

- Start from an initial state X_0 .
- Repeat until satisfied:
 - Generate the set of neighbours of X_i and evaluate them.
 - Select one of the neighbours, X_{i+1} .
 - The selected neighbor becomes the new state.

Important questions:

1. How do we choose the set of neighbours to consider?
 2. How do we select one of the neighbours?
- Defining the set of neighbours is a *design choice* and has crucial impact on performance.

Generic constructive search algorithm

Initialize the search tree with the initial state s_0 as root

Repeat:

 If no node can be expanded:

 return Failure

Choose a node for expansion, using some search strategy

 If the node is the goal:

 return Corresponding Path

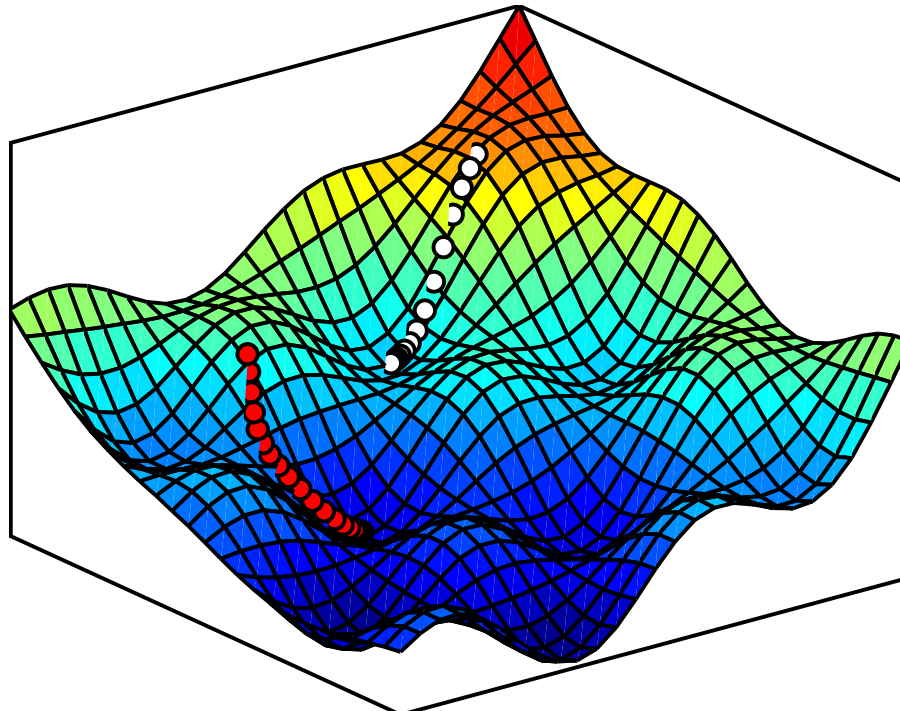
 Else:

 For each applicable action a at the state s of the node:

 Apply the transition $(s, a) \rightarrow s'$ and add s' to the tree

What moves should we consider?

- **Example 1:** Search for high or low ground
 - Start with initial state = random position.
 - Move to an adjacent position.
 - Terminate when goal is reached.



What moves should we consider?

- **Example 1:** Search for high or low ground
 - Start with initial state = random position.
 - Move to an adjacent position.
 - Terminate when goal is reached.
- **Example 2:** Traveling Salesman Problem
 - Start with initial state = a random (possibly incomplete/illegal) tour.
 - Swap cities to obtain a new partial tour.
 - Terminate when constraints are met.

Hill climbing

- Also called **greedy local search**
- In continuous state space, related to **gradient ascent**

Start from an initial configuration X_0 with value $E(X_0)$

$X \leftarrow X_0$, and $E \leftarrow E(X_0)$

Repeat until satisfied:

Generate the set of neighbours of X_i and their value $E(X_i)$.

Let $E_{max} = \max_i E(X_i)$ be the value of the **best** successor,

$i^* = \operatorname{argmax}_i E(X_i)$ be the index of the **best** successor.

if $E_{max} \leq E$:

return X (we are at an optimum)

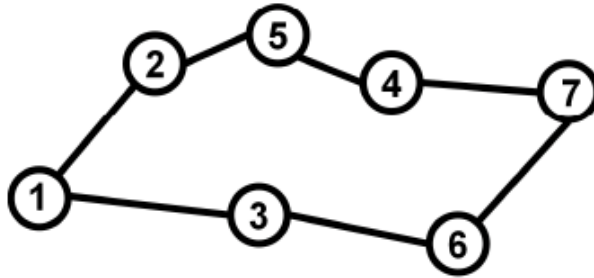
else:

let $X \leftarrow X_{i^*}$, and $E \leftarrow E_{max}$. (take a greedy step)

Properties of hill climbing

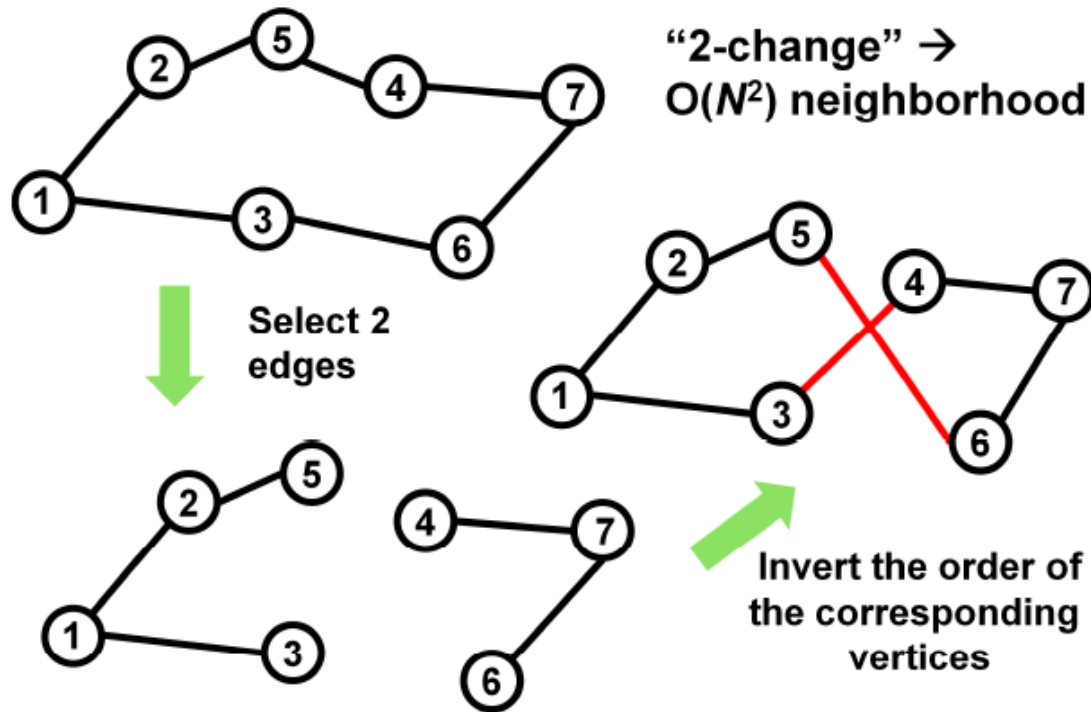
- Very popular in AI:
 - Trivial to program!
 - Requires no memory of where we've been (no backtracking).
 - Can handle very large problems.
- Neighbourhood function is important
 - **Small neighbourhood:** fewer neighbours to evaluate, but possibly worse solution.
 - **Large neighbourhood:** more computation, but maybe fewer local optima, so better final result.

Example: TSP



- What neighbours should we consider?
- How many neighbours is that?

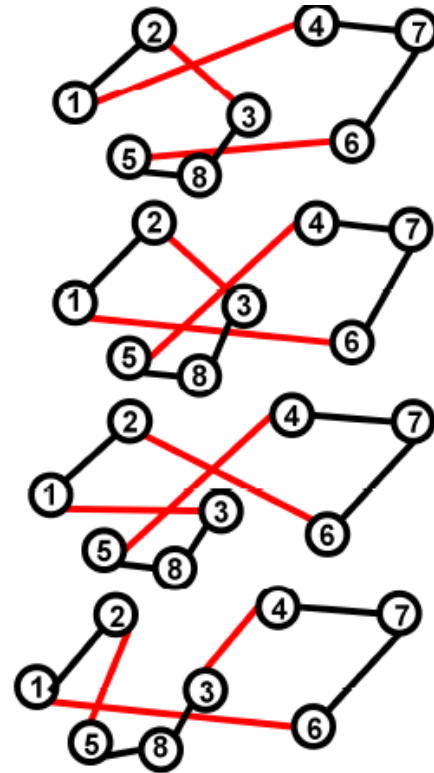
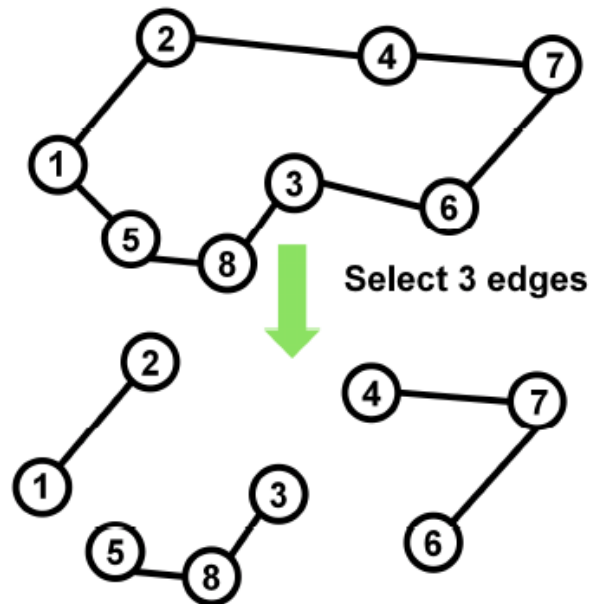
Example: TSP swapping 2 nodes



$O(n^2)$ comes from the fact that we have n edges in a tour, and choose two of them to swap, so there are $\binom{n}{2}$ possible next tours

Example: TSP swapping 3 nodes

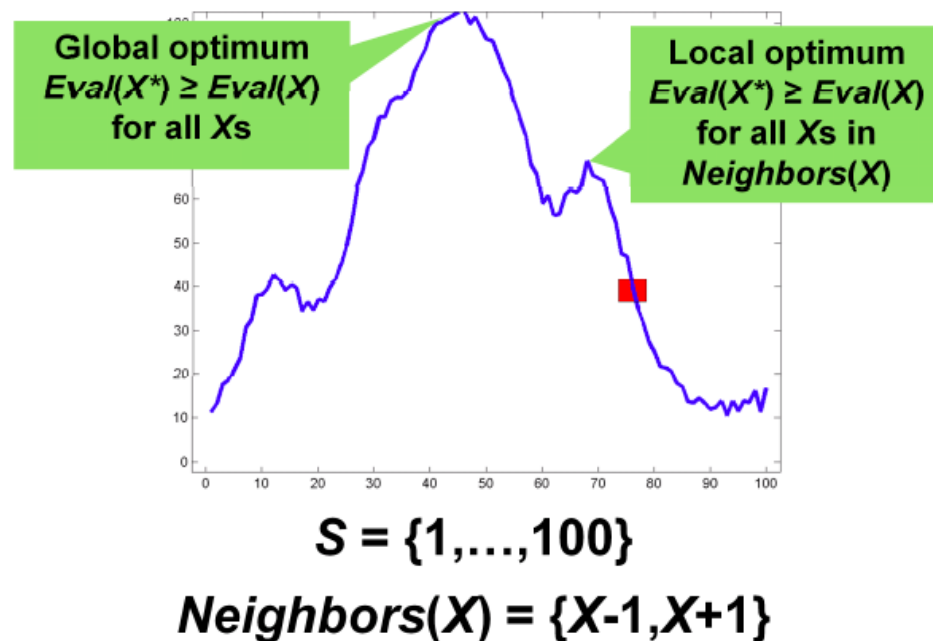
“3-change” $\rightarrow O(N^3)$
neighborhood



There are $\binom{n}{3}$ combinations of edges to choose, and for each set of edges, more than one possible neighbor

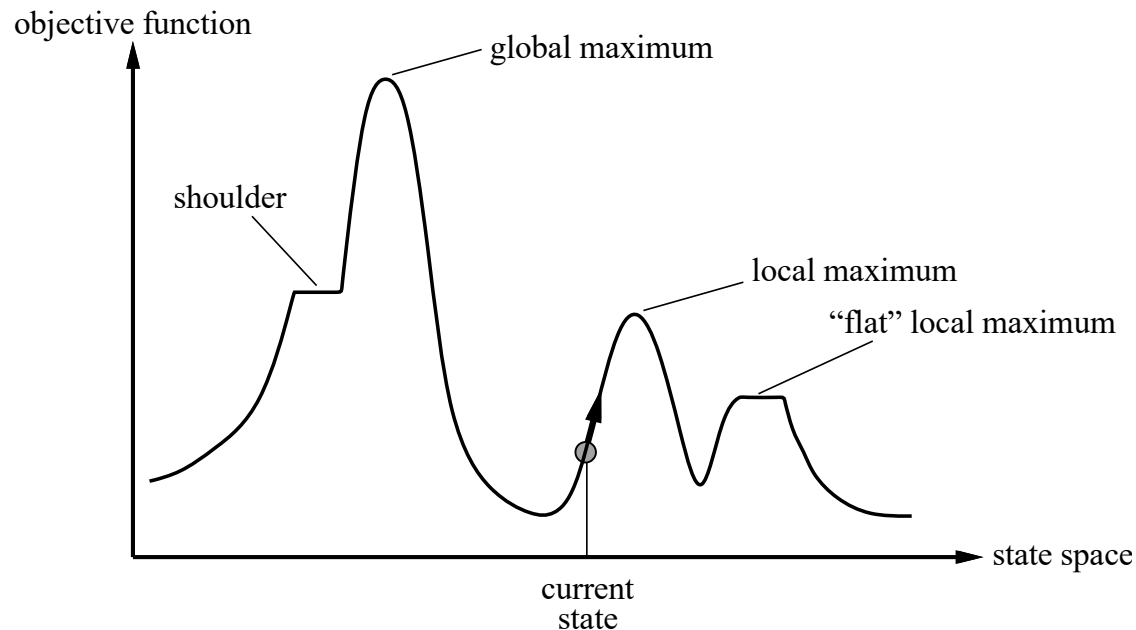
Local vs. Global Optimum

- **Global optimum:** The optimal point over the **full space** of possible configurations.
- **Local optimum:** The optimal point over the **set of neighbours**. One of the (possibly many) optimums.



Problems with hill climbing

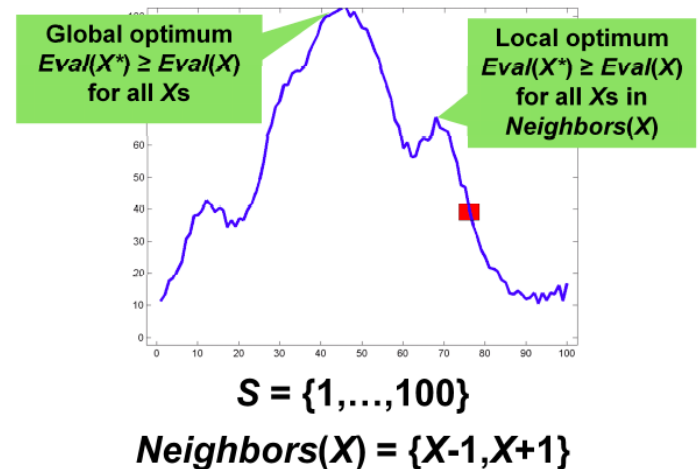
- Can get stuck in a **local maximum** or in a **plateau**



- Depends on the nature of the evaluation function
- A larger neighbourhood function could help
- In some settings, finding a local maximum is good enough!

Improvements to hill climbing

- Quick fix:
 - When stuck in a plateau or local maximum, use random re-starts.
- Slightly better fix:
 - Instead of picking the next best move, **pick any move that produces an improvement**. (Called randomized hill climbing.)
- *But sometimes we need to pick apparently worse moves to eventually reach a better state.*



Simulated annealing



Simulated annealing

Similar to hill climbing, but:

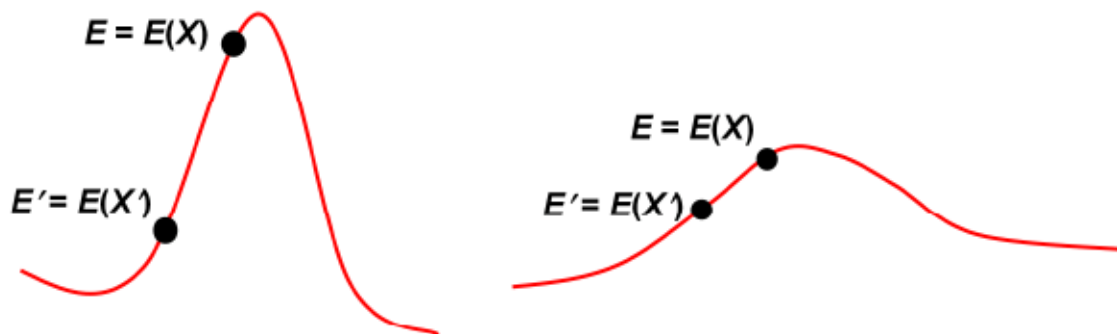
- allows some “bad moves” to try to escape local maxima.
- decrease size and frequency of “bad moves” over time.

Algorithm:

- Start from an initial configuration X_0 with value $E(X_0)$.
 $X \leftarrow X_0$, and $E \leftarrow E(X_0)$
- Repeat until satisfied:
 - Let X_i be a *random neighbour* of X with value $E(X_i)$.
 - If $E_i > E$, let $X_{i*} \leftarrow X_i$ and let $E \leftarrow E_i$ (we found a new better solution).
 - Else, *with some probability p* , still accept the move: $X \leftarrow X_i$ and $E \leftarrow E_i$.
- Return X_{i*} .

What value should we use for p ?

- Many possible choices:
 - A given fixed value.
 - A value that decays to 0 over time.
 - A value that decays to 0, and gives similar chance to “similarly bad” moves.
 - A value that depends on how much worse the bad move is.



What value should we use for p ?

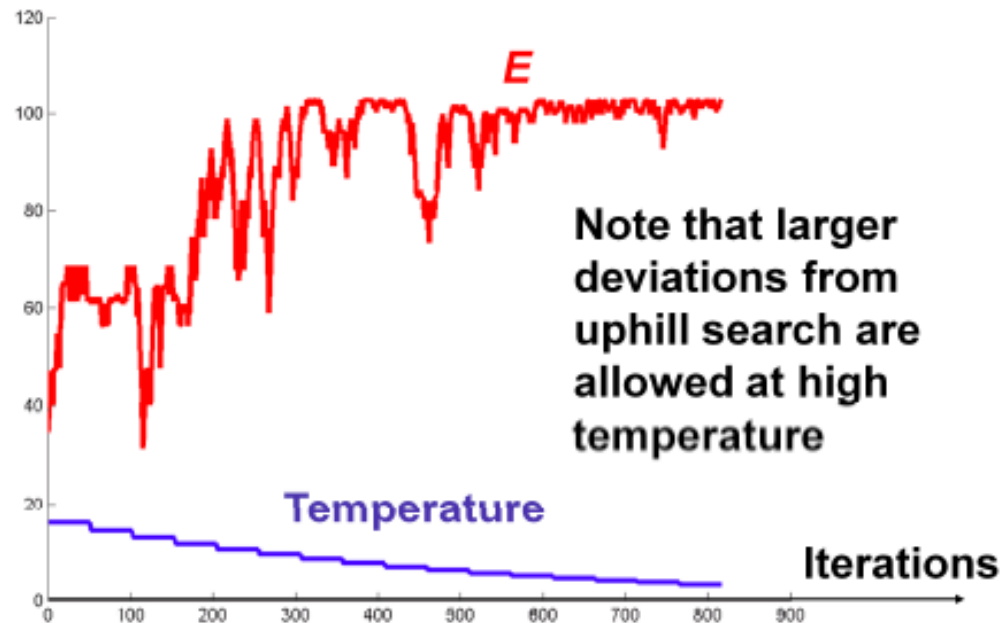
- If the new value E_i is better than the old value E , move to X_i .
- If the new value is worse ($E_i > E$) then move to the neighboring solution with probability: $p = e^{-(E_i - E)/T}$ (**Boltzmann distribution**)
 - If $E - E_i$ is high, p is lower.
 - $T > 0$ is a parameter called the **temperature**, which typically starts high, then decreases over time towards 0.
 - If T is very close to 0, the probability of moving to a worse solution is almost 0.
 - We can gradually decrease T by multiplying by constant $0 < \alpha < 1$ at every iteration.

Properties of simulated annealing

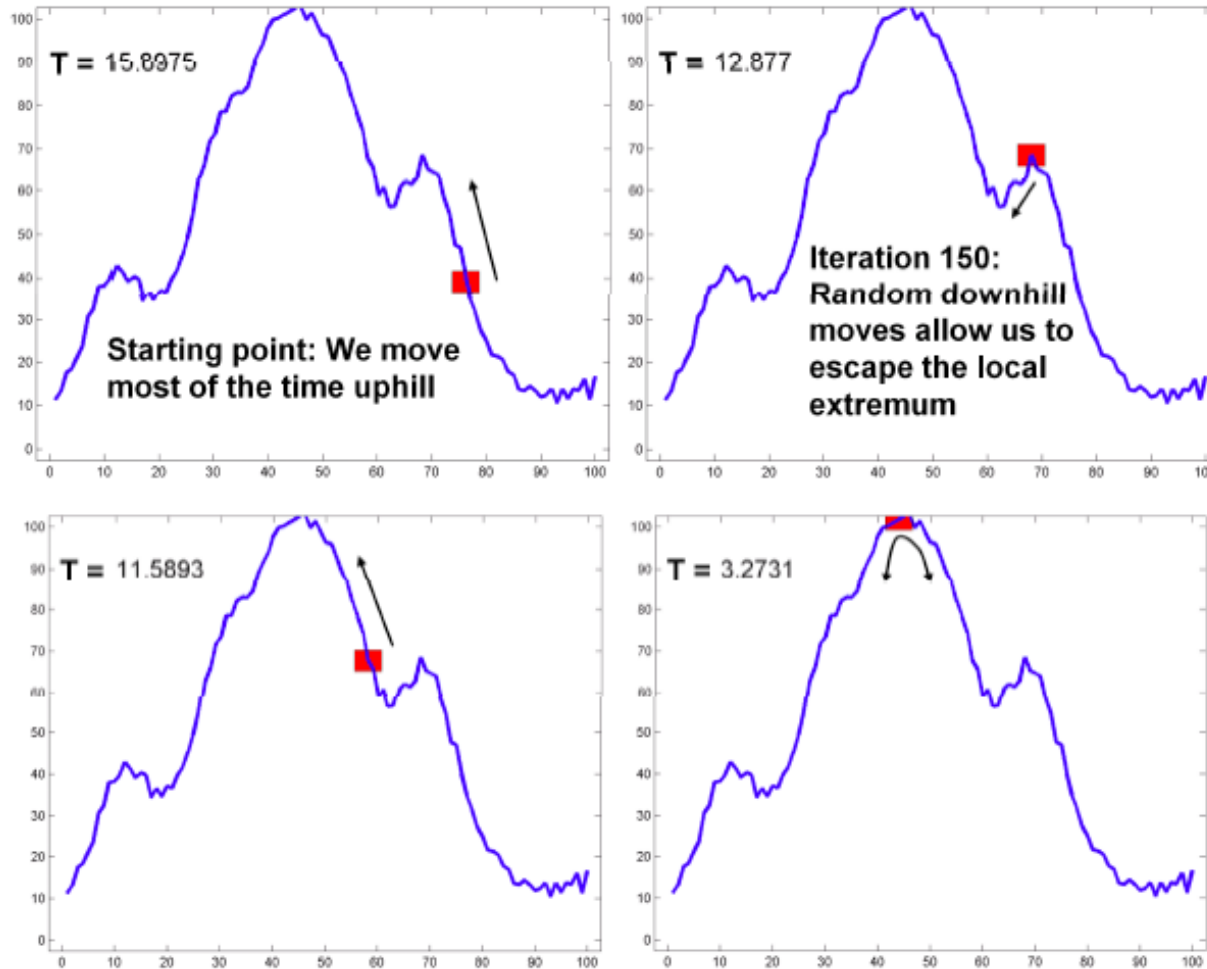
- What happens when T is high?
 - Algorithm is in an **exploratory phase** (even bad moves have a high chance of being picked).
- What happens when T is low?
 - Algorithm is in an **exploitation phase** (the “bad” moves have very low probability).

Properties of simulated annealing

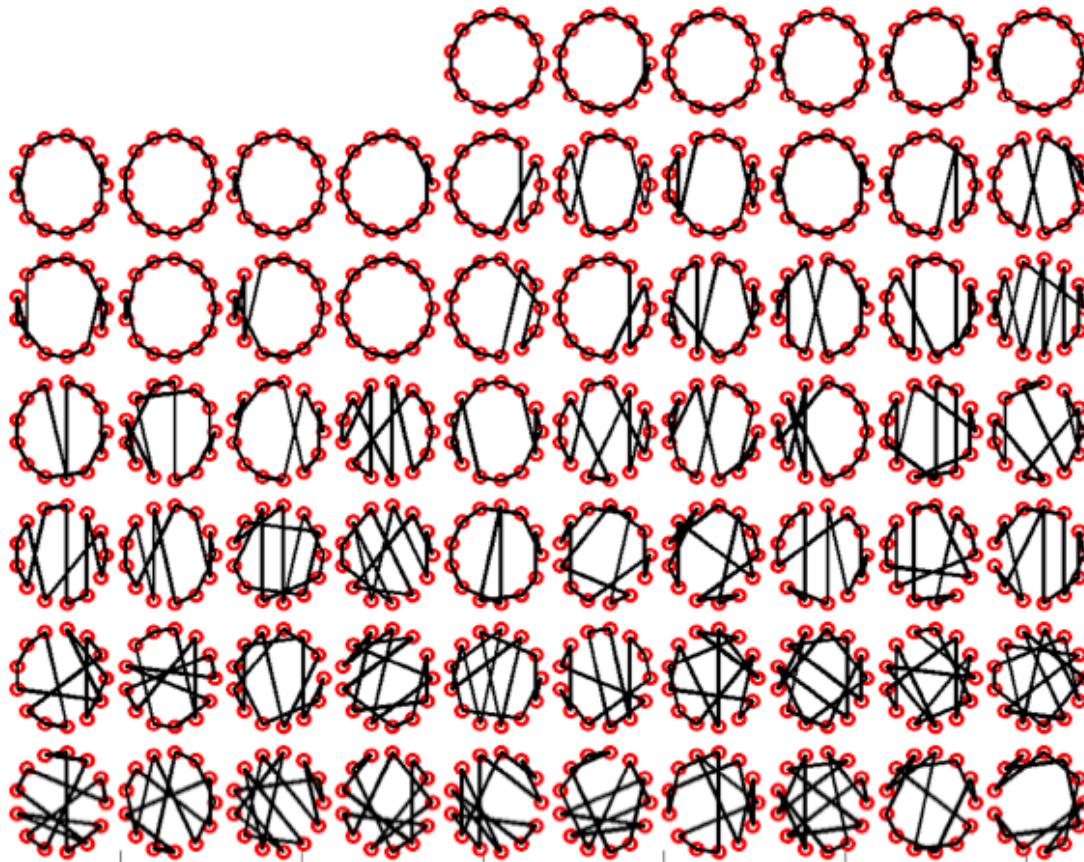
- If T decreases slowly enough, simulated annealing is guaranteed to reach the optimal solution **given infinite time**.
- This result is not practically useful!



Example



TSP example: Searching configurations



The initial configuration is bottom right, final one is top left

Simulated annealing in practice

- Very useful algorithm, used to solve hard optimization problems.
 - e.g., protein design, scheduling large transportation fleets.
- The temperature annealing schedule is crucial (design choice!)
 - Cool too fast: converge to sub-optimal solution.
 - Cool too slow: don't converge.
- Simulated annealing is an example of a randomized search or a **Monte Carlo** search method.
 - **Basic idea:** random sampling, possibly with a heuristic, instead of systematically sweeping (because sweeping is difficult or impossible)

Mitigating the local optimum problem

- Even simulated annealing can get stuck in local maxima!
- More strategies to find a good solution:
 - Beam search
 - Genetic algorithms
 - Parallel search

Local beam search

- Keep track of k states rather than just one state
- Algorithm:
 - Begin with k randomly generated states
 - At each step, generate all the neighbours of the k states
 - If one is a goal, terminate and return that state
 - Otherwise, keep the top k solutions across all the neighbours, discard the remaining
 - k is called the **beam width**
- This might feel similar to a random-restart search
 - conduct a series of searches from randomly generated initial states
- But here information is *shared* among search threads

Local beam search schematic

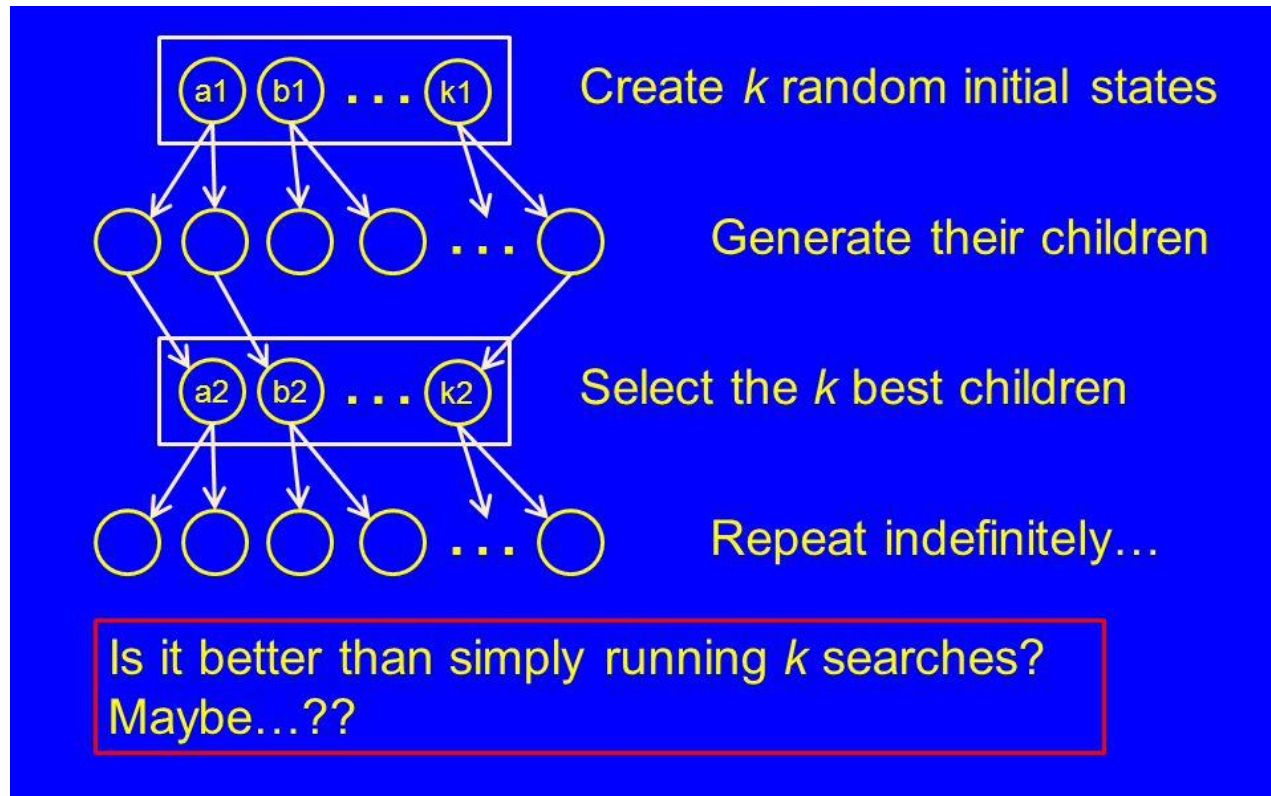


Image source: Sarah Strickland, <http://slideplayer.com/slide/4897669/>

Evolutionary computing

- Refers generally to computational procedures patterned after biological evolution
- Many solutions (individuals) exist
- Nature looks for the best individual (i.e., the fittest)
- Evolutionary search procedures are also parallel, probabilistically perturbing several potential solutions

Genetic algorithms

- A candidate solution is called an **individual**.
 - In a traveling salesman problem, an individual is a tour
- Each individual has a **fitness** scored by a **fitness function**.
- A set of individuals is called a **population**.
- Populations change over **generations**, by applying **operations** to individuals.
 - **operations** = {mutation, crossover, selection}
 - probability of selection is proportional to fitness score
- Individuals with higher fitness are more likely to survive & reproduce.
- Individual can be represented by a binary string:
 - Allows operations to be carried out easily.

Mutation

- A way to generate desirable features that are not present in the original population by injecting random change.
 - Typically, mutation just means changing a 0 to a 1 (and vice versa).
- The mutation rate controls probab. of mutation occurring
- We can allow mutation in all individuals, or just in the offspring.

1	0	0	1	1	0	0
---	---	---	---	---	---	---

1	0	0	1	0	0	0
---	---	---	---	---	---	---

1	1	1	1	1	0	0
---	---	---	---	---	---	---

Select a
random
individual

1	1	1	1	1	0	0
---	---	---	---	---	---	---

Select a
random
entry

1	1	0	1	1	0	0
---	---	---	---	---	---	---

Change
that entry

Crossover

- Combine parts of individuals to create new individuals
- Single-point crossover:
 - Choose a crossover point, cut individuals there, swap the pieces.

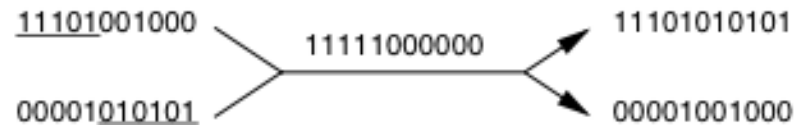
E.g. $\begin{array}{ccc} 101 | 0101 & \xrightarrow{\quad} & 101 | 1110 \\ 011 | 1110 & \xrightarrow{\quad} & 011 | 0101 \end{array}$

- Implementation:
 - Use a crossover mask, which is a binary string
e.g., mask = 1110000
 - At position i , take from first parent if $mask_i$ is 1, otherwise 0
- Multi-point crossover can be implemented with arbitrary mask

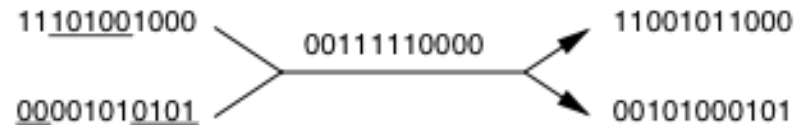
Encoding operators as binary masks

Initial strings Crossover Mask Offspring

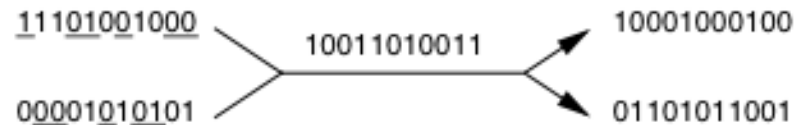
Single-point crossover:



Two-point crossover:



Uniform crossover:



Point mutation:



Typical genetic algorithm

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$child \leftarrow$ REPRODUCE(x, y)

if (small random probability) **then** $child \leftarrow$ MUTATE($child$)

 add $child$ to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

Selection: Survival of the fittest

- As in natural evolution, fittest individuals are more likely to survive.

- Several ways to implement this idea:

1. Fitness proportionate selection:

$$P(i) = \frac{Fitness(i)}{\sum_{j=1}^p Fitness(j)}$$

Can lead to crowding (multiple copies being propagated).

2. Tournament selection:

Pick i, j at random with uniform probability. With prob p select the fitter one. Only requires comparing two individuals.

3. Rank selection:

Sort all hypothesis by fitness. Probability of selection is proportional to rank.

4. Softmax (Boltzmann) selection: $P(i) = \frac{e^{Fitness(i)/T}}{\sum_{j=1}^p e^{Fitness(j)/T}}$

- **Elitist variant:** Copy the best solution(s) directly to the next generation

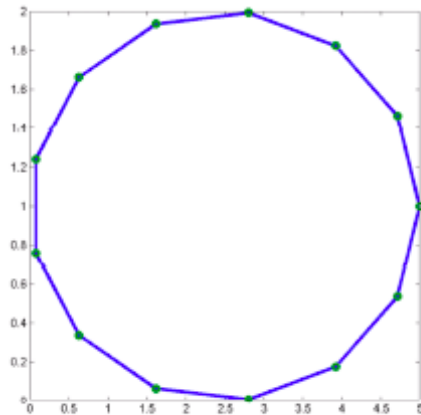
Genetic algorithms as search

- **States:** possible solutions
- **Search operators:** mutation, crossover, selection
- Relation to previous search algorithms:
 - Beam search, since several solutions are maintained in parallel
 - Hill-climbing on the fitness function
 - Mutation and crossover should allow us to get out of local minima

Example: Solving TSP with a GA

- Each individual is a tour.
- Mutation swaps a pair of edges (many other operations are possible and have been tried in literature.)
- Crossover cuts the parents in two and swaps them. Reject any invalid offsprings.
- Fitness is the length of the tour.
- Note that GA operations (crossover and mutation) described here are fancier than the simple binary examples given before.

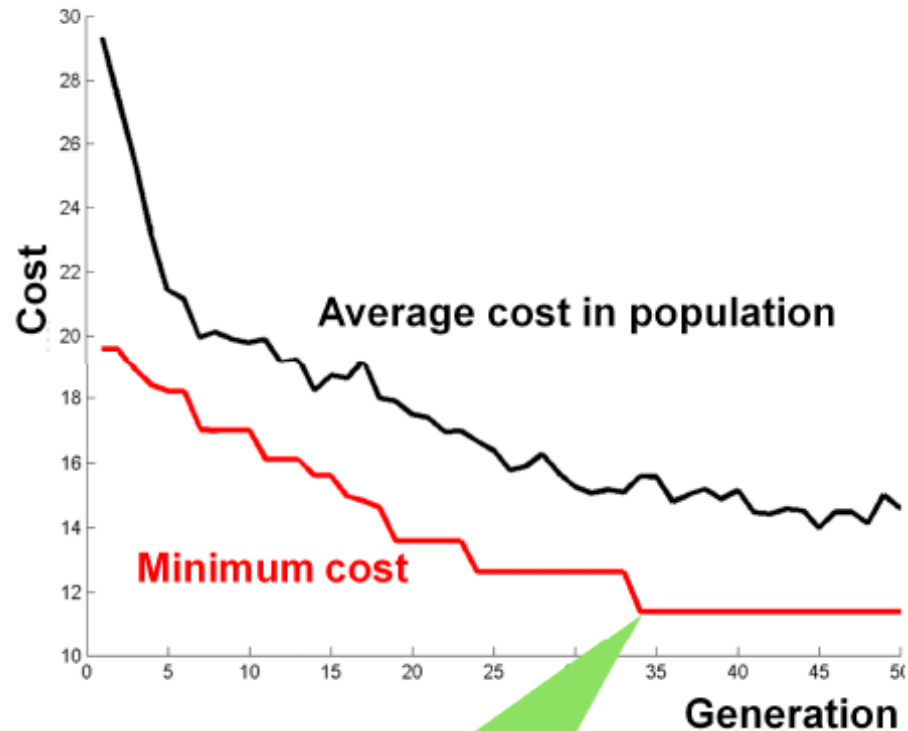
Example: Solving TSP with a GA



$N = 13$

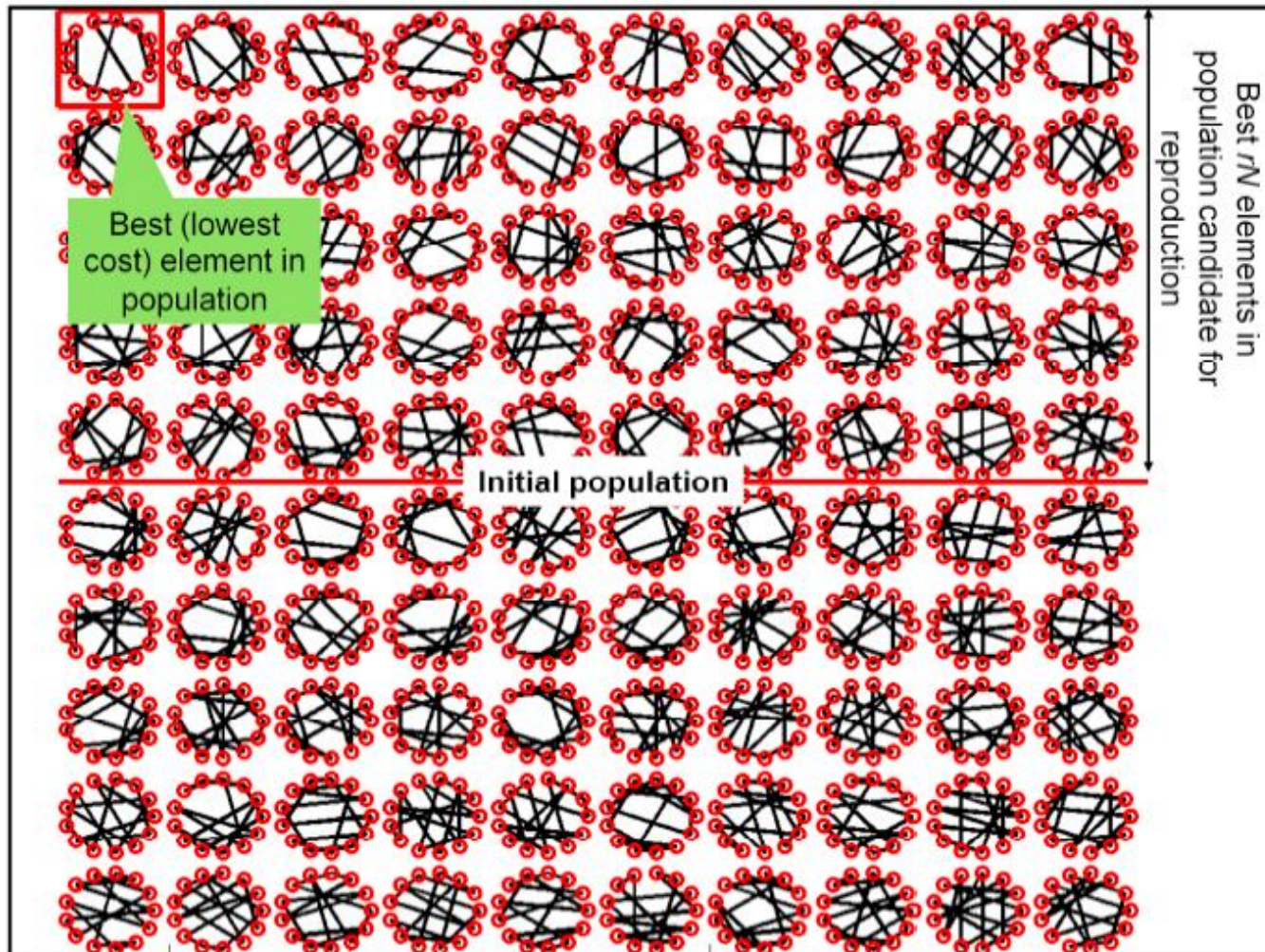
$P = 100$ elements in population

$\mu = 4\%$ mutation rate
 $r = 50\%$ reproduction rate

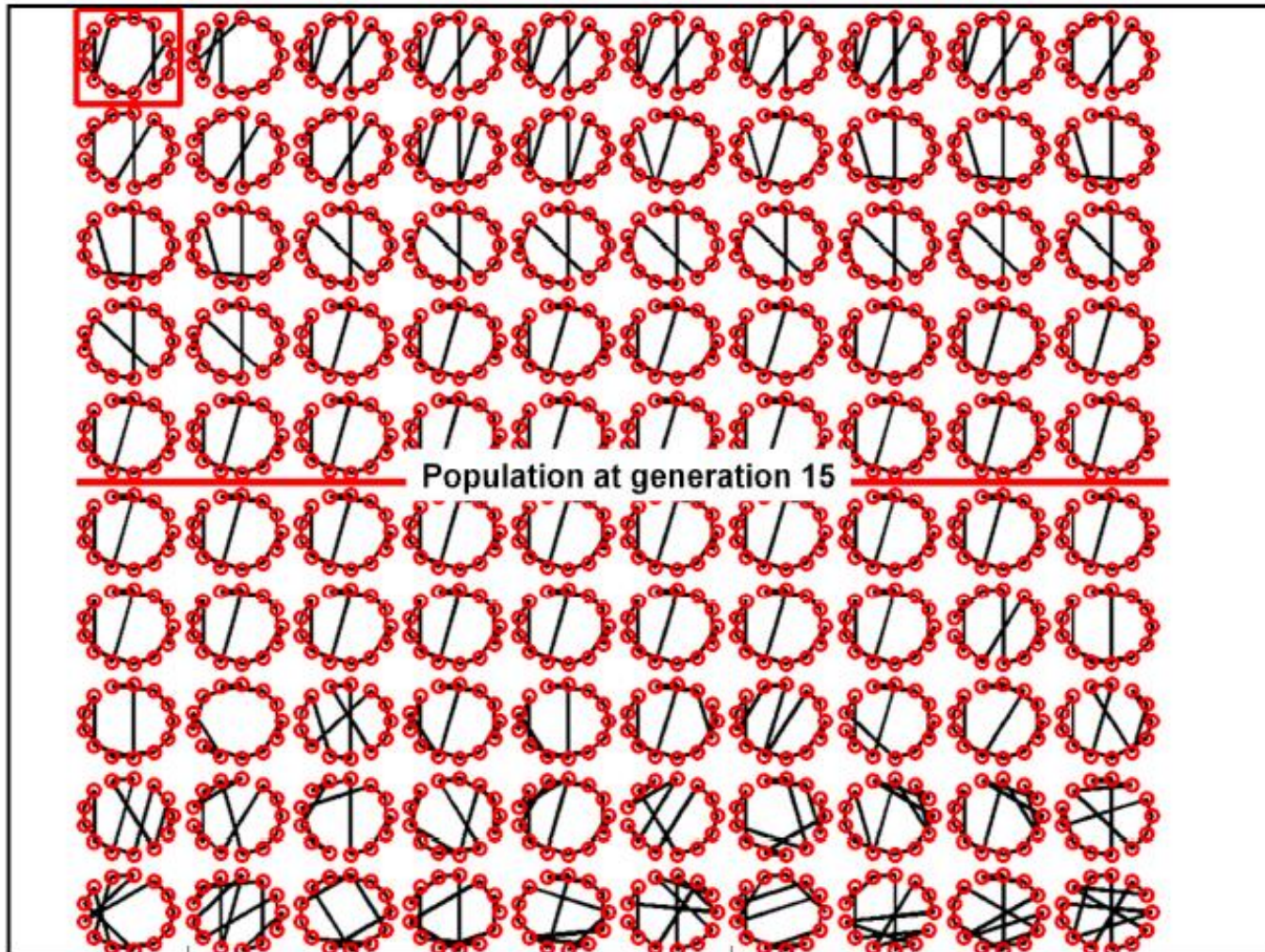


Optimal solution reached at generation 35

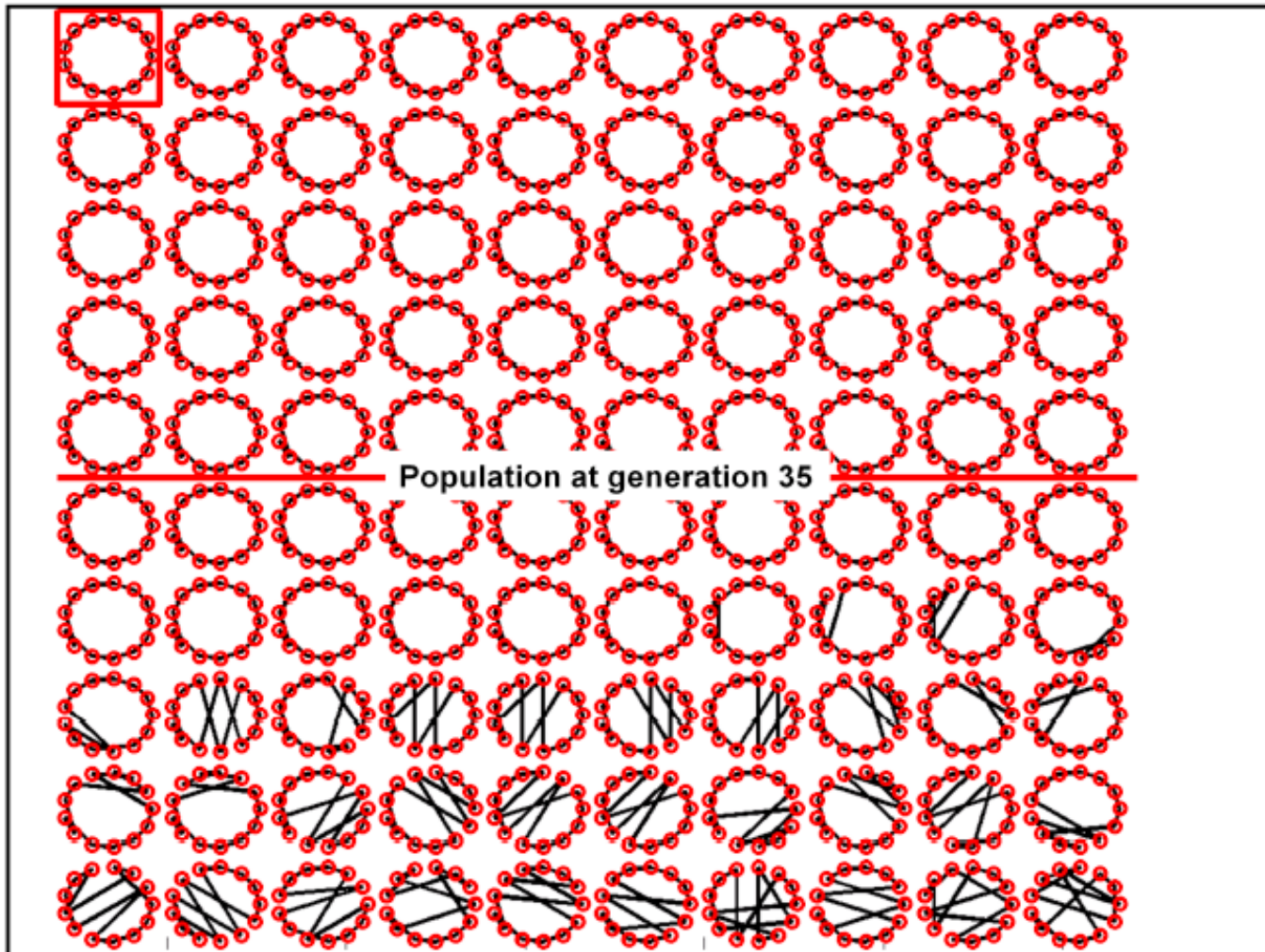
TSP example: Initial generation



TSP example: Generation 15



TSP example: Generation 30



Example: Program repair

```
1 void zunebug(int days) {
2     int year = 1980;
3     while (days > 365) {
4         if (isLeapYear(year)) {
5             if (days > 366) {
6                 days -= 366;
7                 year += 1;
8             }
9             else {
10                }
11        }
12        else {
13            days -= 365;
14            year += 1;
15        }
16    }
17    printf("current year is %d\n", year);
18 }
```

Example: Program repair

- Each individual is an abstract syntax tree and a weighted path through the tree



Example: Program repair

- Each individual is an abstract syntax tree and a weighted path through the tree
 - weighted paths contain statements visited on the negative test case
- Fitness: how well the program avoids the bug (negative test case) and does everything else it is supposed to (positive test cases)
- Mutation: delete from, swap, insert statement in weighted path
- Crossover: individual is crossed with original parent program; crossover point is in weighted path

Original: $Pre \circ O1 \circ O2 \circ Post$

Variant: $Pre \circ V1 \circ V2 \circ Post$

Children: $Pre \circ O1 \circ V2 \circ Post, Pre \circ V1 \circ O2 \circ Post$

The good and bad of GAs

- Good:
 - Intuitively appealing, due to evolution analogy.
 - If tuned right, can be very effective (good solution with few steps).
- Bad:
 - Performance depends crucially on the problem encoding. Good encodings are difficult to find!
 - Many parameters to tweak! Bad parameter settings can result in very slow progress, or the algorithm is stuck in local minima.
 - With mutation rate is too low, can get overcrowding (many copies of the identical individuals in the population).

Parallel search

- Run many separate searches (hill climbing or simulated annealing) in parallel.
- Keep the best solution found.
- Search speed can be greatly improved by using many processors (including, most recently, GPUs).

Summary

- Two distinctions drawn compared to previously:
 - Problem: Often, interested in best state according to *evaluation function* (*optimization* problem); less interested in *path* to that state
 - Search strategy: Start with some initial solution, then employ some strategy to gradually improve it; not searching systematically
- Optimization problems are widespread and important.
- It is often infeasible to enumerate lots of solutions.
- Goal is to get a reasonable (not necessarily optimal) solution.

Summary

- Apply a **local search** and move in a promising direction.
 - **Hill climbing** always moves in the (locally) best direction
 - **Simulated annealing** allows some moves towards worse solutions
 - Parallelism and **beam search** can be exploited to improve results; find a better local optimum
 - **Genetic algorithms** are an alternative related to simulated annealing which has an analogy to biological evolution